

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

03

자바의 기초 - 자료형



03-8 맵

03-9 집합

03-8 맵

- **맵(map)**
  - 대응 관계를 쉽게 표현할 수 있게 해주는 자료형
  - 사전(dictionary)과 비슷

키(key)	값(value)
people	사람
baseball	야구

  - 리스트나 배열처럼 순차적으로(sequential) 요솟값을 구하지 않고 키(key)를 이용해 값(value)을 얻는다

03-8 맵

HashMap

put

- key와 value를 추가할 수 있음

Do it! 코딩해 보세요

```
import java.util.HashMap;

public class Sample {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("people", "사람");
        map.put("baseball", "야구");
    }
}
```

03-8 맵

HashMap

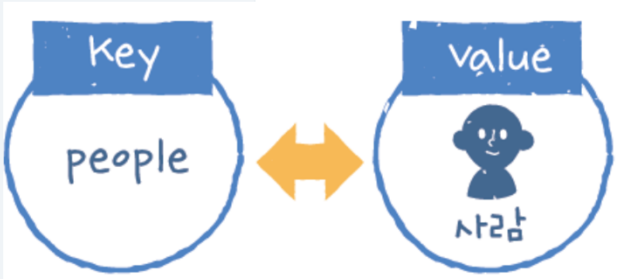
get

- key에 해당하는 value를 얻을 때 사용

Do it! 코딩해 보세요

```
import java.util.HashMap;

public class Sample {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("people", "사람");
        map.put("baseball", "야구");
        System.out.println(map.get("people")); // "사람" 출력
    }
}
```



실행 결과

```
사람
```

03-8 맵

HashMap

containsKey

- 맵에 해당 key가 있는지를 참(true) 또는 거짓(false)으로 리턴

Do it! 코딩해 보세요

```
(... 생략 ...)
HashMap<String, String> map = new HashMap<>();
map.put("people", "사람");
map.put("baseball", "야구");
System.out.println(map.containsKey("people")); // true 출력
```

실행 결과

```
true
```

03-8 맵

HashMap

remove

- 해당 key의 항목을 삭제한 후 value 값을 리턴

Do it! 코딩해 보세요

```
(... 생략 ...)
HashMap<String, String> map = new HashMap<>();
map.put("people", "사람");
map.put("baseball", "야구");
System.out.println(map.remove("people")); // "사람" 출력
```

실행 결과

```
사람
```

03-8 맵

HashMap

size

- 맵 요소의 개수를 리턴

Do it! 코딩해 보세요

```
(... 생략 ...)
HashMap<String, String> map = new HashMap<>();
map.put("people", "사람");
map.put("baseball", "야구");
System.out.println(map.remove("people")); // 사람 출력
System.out.println(map.size()); // 1 출력
```

실행 결과

```
사람
1
```

03-8 맵

HashMap

keySet

- 맵의 모든 key를 모아서 리턴

Do it! 코딩해 보세요

```
import java.util.HashMap;

public class Sample {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("people", "사람");
        map.put("baseball", "야구");
        System.out.println(map.keySet());  // [baseball, people] 출력
    }
}
```

실행 결과

```
[baseball, people]
```

03-9 집합

집합(set)

- 집합과 관련된 것을 쉽게 처리하기 위해 만든 것
- HashSet, TreeSet, LinkedHashSet 등이 있음

Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>(Arrays.asList("H", "e", "l", "l", "o"));
        System.out.println(set);  // [e, H, l, o] 출력
    }
}
```

실행 결과

```
[e, H, l, o]
```

03-9 집합

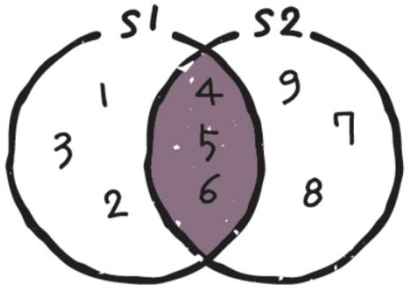
- **집합 자료형의 2가지 특징**
  - 중복을 허용하지 않는다.
  - 순서가 없다(unordered).
    - 인덱싱으로 값을 얻을 수 없음
    - 맵 자료형과 비슷

03-9 집합

▪ 교집합, 합집합, 차집합 구하기

▪ 교집합 구하기

- s1과 s2의 교집합
- retainAll 메서드 이용



Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<Integer> s1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5, 6));
        HashSet<Integer> s2 = new HashSet<>(Arrays.asList(4, 5, 6, 7, 8, 9));

        HashSet<Integer> intersection = new HashSet<>(s1); // s1으로 intersection 생성
        intersection.retainAll(s2); // 교집합 수행
        System.out.println(intersection); // [4, 5, 6] 출력
    }
}
```

실행 결과

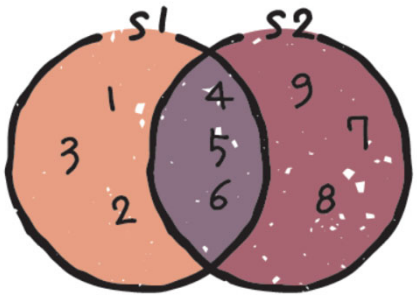
```
[4, 5, 6]
```

03-9 집합

▪ 교집합, 합집합, 차집합 구하기

▪ 합집합 구하기

- s1과 s2의 합집합
- addAll 메서드 사용



Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<Integer> s1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5, 6));
        HashSet<Integer> s2 = new HashSet<>(Arrays.asList(4, 5, 6, 7, 8, 9));

        HashSet<Integer> union = new HashSet<>(s1);  // s1으로 union 생성
        union.addAll(s2); // 합집합 수행
        System.out.println(union);  // [1, 2, 3, 4, 5, 6, 7, 8, 9] 출력
    }
}
```

실행 결과

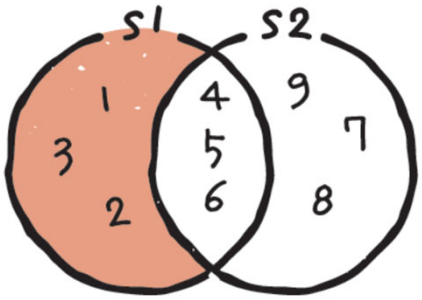
```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

03-9 집합

▪ 교집합, 합집합, 차집합 구하기

▪ 차집합 구하기

- s1과 s2의 합집합
- addAll 메서드 사용



Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<Integer> s1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5, 6));
        HashSet<Integer> s2 = new HashSet<>(Arrays.asList(4, 5, 6, 7, 8, 9));

        HashSet<Integer> substract = new HashSet<>(s1);  // s1으로 substract 생성
        substract.removeAll(s2); // 차집합 수행
        System.out.println(substract);  // [1, 2, 3] 출력
    }
}
```

실행 결과

```
[1, 2, 3]
```

03-9 집합

▪ 집합 자료형과 관련된 메서드 — add, addAll, remove

▪ **add** - 집합 자료형에 값을 추가할 때 사용

Do it! 코딩해 보세요

```
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Jump");
        set.add("To");
        set.add("Java");
        System.out.println(set);  // [Java, To, Jump] 출력
    }
}
```

실행 결과

```
[Java, To, Jump]
```

03-9 집합

▪ 집합 자료형과 관련된 메서드 — add, addAll, remove

▪ **addAll** - 값을 한꺼번에 여러 개 추가할 때 사용

Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();
        set.add("Jump");
        set.addAll(Arrays.asList("To", "Java"));
        System.out.println(set);  // [Java, To, Jump] 출력
    }
}
```



실행 결과

```
[Java, To, Jump]
```

03-9 집합

▪ 집합 자료형과 관련된 메서드 — add, addAll, remove

▪ **remove** - 특정 값을 제거할 때 사용

Do it! 코딩해 보세요

```
import java.util.Arrays;
import java.util.HashSet;

public class Sample {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>(Arrays.asList("Jump", "To", "Java"));
        set.remove("To");
        System.out.println(set);  // [Java, Jump] 출력
    }
}
```

실행 결과

```
[Java, Jump]
```



감사합니다.